

Módulo 4. Redes neuronales con Keras

4.1. Introducción acerca de las redes neuronales

El hombre se ha caracterizado siempre por su búsqueda constante de nuevas vías para mejorar sus condiciones de vida. Estos esfuerzos le han servido para reducir el trabajo en aquellas operaciones en las que la fuerza juega un papel primordial. Los progresos obtenidos han permitido dirigir estos esfuerzos a otros campos, como por ejemplo, a la construcción de máquinas calculadoras que ayuden a resolver de forma automática y rápida determinadas operaciones que resultan tediosas cuando se realizan a mano.

Uno de los primeros en acometer esta empresa fue Charles Babbage, quien trató infructuosamente de construir una máquina capaz de resolver problemas matemáticos. Posteriormente otros tantos intentaron construir máquinas similares, pero no fue hasta la Segunda Guerra Mundial, cuando ya se disponía de instrumentos electrónicos, que se empezaron a recoger los primeros frutos. En 1946 se construyó la primera computadora electrónica, ENIAC. Desde entonces los desarrollos en este campo han tenido un auge espectacular.

Estas máquinas permiten implementar fácilmente algoritmos para resolver una gran diversidad de problemas que antes resultaban engorrosos de resolver. Sin embargo, se observa una limitación importante: ¿qué ocurre cuando el problema que se quiere resolver no admite un tratamiento algorítmico, como es el caso, por ejemplo, de la clasificación de objetos por rasgos comunes? Este ejemplo demuestra que la construcción de nuevas máquinas más versátiles requiere un enfoque del problema desde otro punto de vista. Los desarrollos actuales de los científicos se dirigen al estudio de las capacidades humanas como una fuente de nuevas ideas para el diseño de las nuevas máquinas. Así, la inteligencia artificial es un intento por descubrir y describir aspectos de la inteligencia humana que pueden ser simulados mediante máquinas. Esta disciplina se ha desarrollado fuertemente en los últimos años teniendo aplicación en algunos campos como visión artificial, demostración de teoremas, procesamiento de información expresada mediante lenguajes humanos, etc.

Las redes neuronales son más que otra forma de emular ciertas características propias de los humanos, como la capacidad de memorizar y de asociar hechos. Si se examinan con atención aquellos problemas que no pueden expresarse a través de un algoritmo, se observará que todos ellos tienen una característica en común: la experiencia. El hombre es capaz de resolver estas situaciones acudiendo a la experiencia acumulada. Así, parece claro que una forma de aproximarse al problema consiste en la construcción de sistemas que sean capaces de reproducir esta característica humana. En definitiva, las redes neuronales no son más que un modelo artificial y simplificado del cerebro humano, que es el ejemplo más perfecto del que disponemos para un sistema que es capaz de adquirir conocimiento a través de la experiencia. Una red neuronal es “un nuevo sistema

para el tratamiento de la información, cuya unidad básica de procesamiento está inspirada en la célula fundamental del sistema nervioso humano: la neurona”.

Como ya se sabe, el pensamiento tiene lugar en el cerebro, que consta de billones de neuronas interconectadas. Así, el secreto de la “inteligencia”, sin importar cómo se defina, se sitúa dentro de estas neuronas interconectadas y de su interacción. También, es bien conocido que los humanos son capaces de aprender. Por otra parte, aprendizaje significa que aquellos problemas que inicialmente no pueden resolverse, pueden ser resueltos después de obtener más información acerca del problema. Por lo tanto, las redes neuronales...

- Consisten de unidades de procesamiento que intercambian datos o información.
- Se utilizan para reconocer patrones, incluyendo imágenes, manuscritos y secuencias de tiempo (por ejemplo: tendencias financieras).
- Tienen capacidad de aprender y mejorar su funcionamiento.

4.2. Un poco de historia sobre las redes neuronales

El mundo de las redes neuronales está en auge. Poder simular el cerebro humano en un ordenador parece ser uno de los hitos más prometedores del área de la informática. Es bien sabido que este hito aún no se ha conseguido de forma completa, pero mediante algoritmos de Machine Learning, ya es posible entrenar máquinas para que aprendan de forma parecida a como lo haría nuestro cerebro.

Desde hace cientos de años, se ha estudiado el cerebro humano. Sin embargo, a partir de los años 50, cuando se empezó a desarrollar las bases de la tecnología actual, hubo un fuerte avance en este estudio desde la perspectiva de la computación por medio de las redes neuronales artificiales. Tal como su nombre lo indica, pretenden imitar la forma de funcionamiento de las neuronas que conforman el cerebro humano.

Años 40:

- Alan Turing, en 1936, fue el primero en estudiar el cerebro como una forma de ver el mundo de la computación. Sin embargo, los primeros teóricos que concibieron los fundamentos de la computación neuronal fueron el neurólogo Warren McCulloch y el matemático Walter Pitts, quienes, en 1943, propusieron los primeros modelos matemáticos y eléctricos de redes neuronales. Describieron cómo funcionaban las neuronas y modelaron una red neuronal simple utilizando circuitos eléctricos.
- A finales de la década, Donald Hebb definió dos conceptos muy importantes:

- El proceso de aprendizaje se localiza principalmente en la sinapsis, también conocida como conexión entre neuronas.
- La información se representa en el cerebro humano mediante un conjunto de neuronas activas o inactivas.

Estas dos reglas se sintetizan en la regla de aprendizaje de Hebb, la cual sigue siendo aplicada en algunos modelos actuales. Esta regla afirma que los cambios en los pesos de las sinapsis se basan en la interacción entre las neuronas pre y post sintácticas, y en el número de veces que se activan de forma simultánea.

Años 50:

- En 1956, en la ciudad de Dartmouth, se llevó a cabo la primera conferencia sobre Inteligencia Artificial donde se discutió sobre la capacidad de las máquinas para simular el aprendizaje humano.
- A finales de la década, el neurobiólogo Frank Rosenblatt empezó a trabajar en el concepto de Perceptrón (su arquitectura está basada en monocapa y tiene la capacidad de aprender a reconocer patrones). Esto dio lugar a la regla de aprendizaje Delta, que permitía emplear señales continuas de entrada y salida.

Años 60:

- En 1960, Bernard Widrow y Marcian Hoff desarrollaron los algoritmos Adaline (Adaptative Linear Elements) y Madaline (Multiple Adaline). Estos modelos fueron empleados para desarrollar la primera solución a un problema real aplicando redes neuronales: un filtro que elimina el eco en las llamadas telefónicas.
- En 1961, Karl Steinbeck, diseñó una red neuronal para simples realizaciones técnicas (memoria asociativa).
- En 1969, casi se produce la muerte abrupta de la redes neuronales, ya que Marvin Minsky y Seymour Papert demostraron matemáticamente que el Perceptron no era capaz de resolver problemas relativamente fáciles, tales como el aprendizaje de una función no lineal. Esto demostró que el Perceptron era muy débil, dado que las funciones no lineales son muy empleadas en el campo de la computación y en diversos problemas del mundo real.

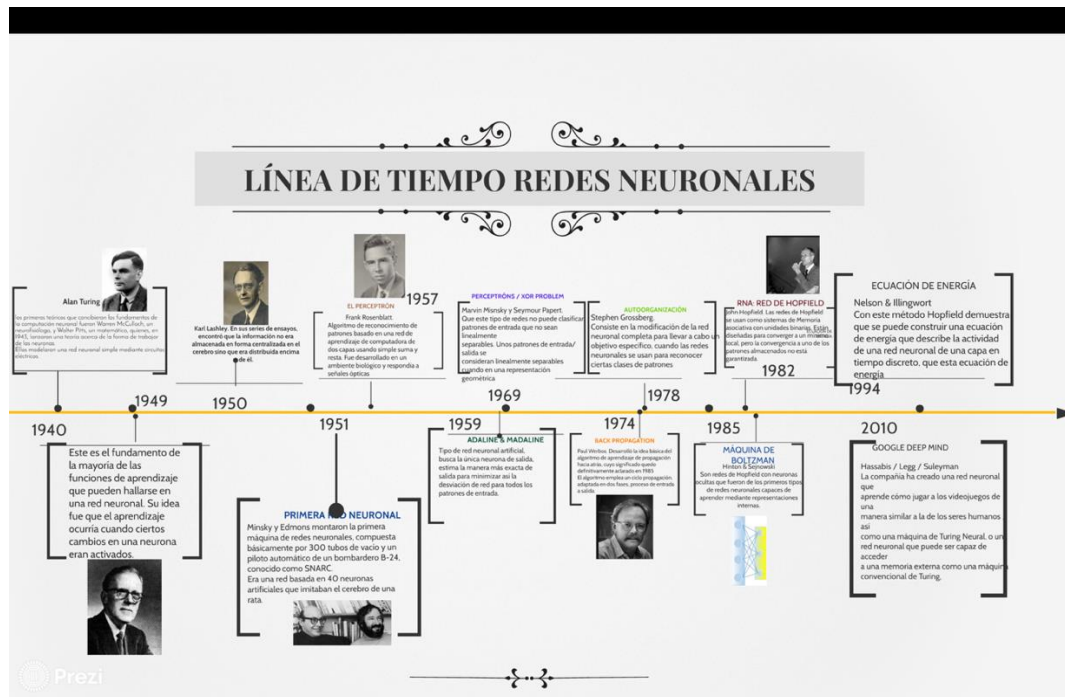
Años 70:

- En 1974, Paul Werbos, desarrolló la idea básica del algoritmo de aprendizaje de propagación hacia atrás (backpropagation); cuyo significado quedó definitivamente aclarado en 1985.
- En 1977, Stephen Grossberg, desarrolló la Teoría de Resonancia Adaptada (TRA), la cual es una arquitectura de red que simula otras habilidades del cerebro: memoria a largo y corto plazo.

Años 80:

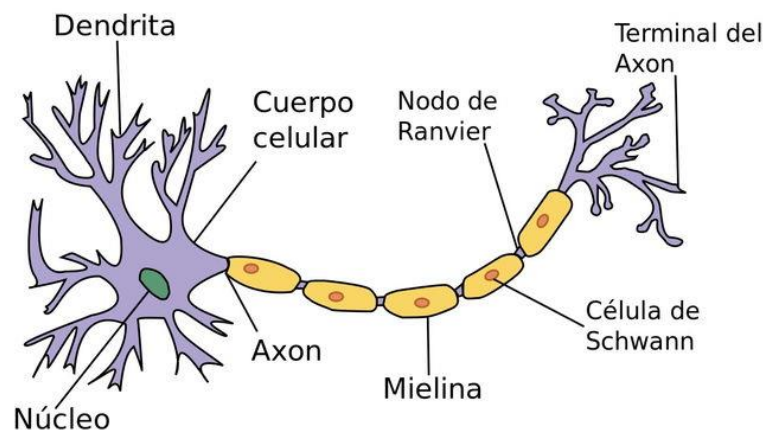
- En 1982, el interés por las redes neuronales se renovó. John Hopfield presentó varios artículos a la Academia Nacional de Ciencias en los que describe el modelado del cerebro humano y además demostró qué aplicaciones podría tener su estudio a la hora de crear nuevos dispositivos.
- En 1984, Kohonen desarrolló una familia de redes de memoria asociativa y mapas auto-organizativos, actualmente llamadas redes de Kohonen.
- En 1985, Paul Werbos desarrolló el algoritmo Backpropagation que resolvió el problema de entrenar redes neuronales multicapas de forma efectiva.
- En 1986, los investigadores Hinton y Sejnowski desarrollaron la máquina de Boltzmann, un tipo de red neuronal recurrente estocástica.

A partir de 1986, el panorama fue alentador con respecto a las investigaciones y el desarrollo de las redes neuronales. En la actualidad, son numerosos los trabajos que se realizan y publican cada año, las aplicaciones nuevas que surgen (sobre todo en el área de control) y las empresas que lanzan al mercado productos nuevos, tanto hardware como software (sobre todo para simulación).



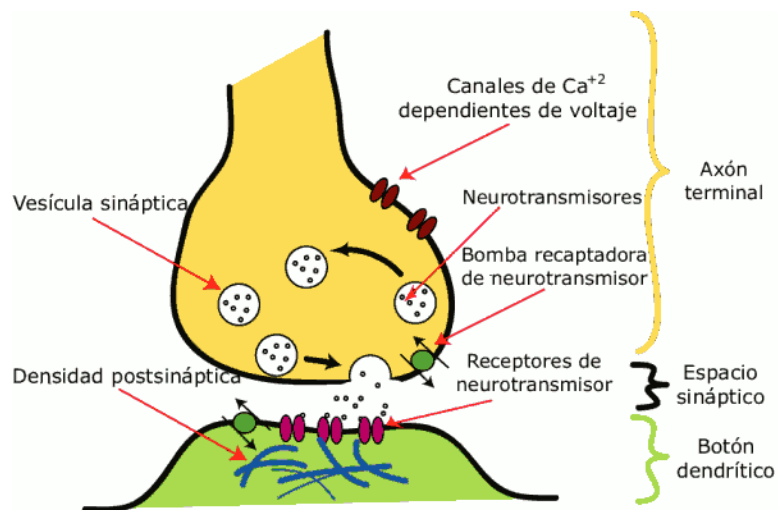
4.3. La neurona biológica vs la neurona artificial

El cerebro visto a un alto nivel y simplificando su estructura, se puede definir como un conjunto de millones de células, llamadas neuronas, interconectadas entre sí mediante sinapsis. La sinapsis se lleva a cabo en la zona donde 2 neuronas se conectan, y las partes de la célula que se encargan de realizarlas son las dendritas y las ramificaciones del axón.



Cada neurona desarrolla impulsos eléctricos que se transmiten a lo largo de ésta mediante su axón. Éste, al final se divide en ramificaciones axonales, que conectan con otras neuronas mediante sus dendritas. El conjunto de elementos que hay entre la ramificación axonal y la dendrita forman la sinapsis, que regula la

transmisión del impulso eléctrico mediante unos elementos químicos llamados neurotransmisores.



Los neurotransmisores liberados en la sinapsis pueden tener un efecto positivo o negativo sobre la transmisión del impulso eléctrico en la neurona que los recibe en sus dendritas. Esta neurona recibe varias señales de las distintas sinapsis, y las combina consiguiendo un cierto nivel de estimulación. En función de este nivel de activación, la neurona emite señales eléctricas mediante impulsos, con una intensidad determinada y con una frecuencia llamada tasa de disparo.

En resumen, si consideramos que la información del cerebro está codificada en impulsos eléctricos que se transmiten entre neuronas, y que los impulsos se ven modificados básicamente en la sinapsis, podemos intuir que la codificación del aprendizaje estará en la sinapsis y en la forma en que las neuronas dejan de pasar o inhiben las señales segregando neurotransmisores.

Por otra parte, las neuronas artificiales son modelos que tratan de simular el comportamiento de las neuronas biológicas. Cada neurona se representa como una unidad de proceso que forma parte de una entidad más grande, la red neuronal.

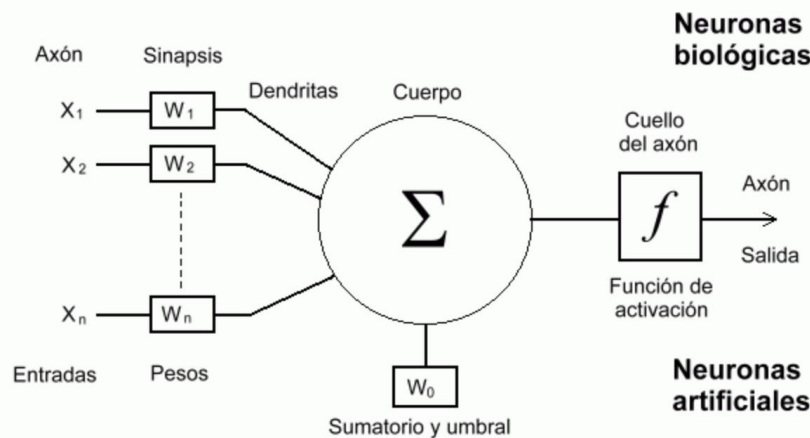
4.4. Definiciones de una red neuronal

Existen numerosas formas de definir a las redes neuronales; desde las definiciones cortas y genéricas hasta las que intentan explicar más detalladamente qué son las redes neuronales. Por ejemplo:

- ✓ Una nueva forma de computación, inspirada en modelos biológicos.
- ✓ Un modelo matemático compuesto por un gran número de elementos procesales organizados en niveles.
- ✓ Un sistema de computación compuesto por un gran número de elementos simples, elementos de procesos muy interconectados, los cuales procesan

información por medio de su estado dinámico como respuesta a entradas externas.

- ✓ Son redes interconectadas masivamente en paralelo de elementos simples (usualmente adaptativos) y con organización jerárquica, las cuales intentan interactuar con los objetos del mundo real del mismo modo que lo hace el sistema nervioso biológico.



Tal como se observa en la imagen anterior, podemos intuir que la neurona artificial se comporta en cierto modo como una neurona biológica, pero de forma simplificada:

- Entradas ($x_1, x_2, \dots$): estos valores pueden ser enteros, reales o binarios y equivaldrían a los impulsos que envían otras neuronas a través de sus dendritas.
- Pesos ($w_1, w_2, \dots$): equivaldrían a los mecanismos de sinapsis necesarios para transmitir el impulso.
- El producto $x_i w_i$ equivaldría a las señales químicas inhibitoras y excitadoras que se dan en la neurona. Estos valores son la entrada de la función de activación, que convierte todo el conjunto de valores en uno solo llamado potencial. La función de ponderación suele ser la suma ponderada de las entradas y los pesos sinápticos.
- La salida de la función de ponderación llega a la función de activación que transforma este valor en otro en el dominio en que trabajan las salidas de las neuronas. Suele ser una función no lineal como la función paso o la función sigmoide, aunque también se pueden usar funciones lineales.

En resumen, podemos definir las siguientes analogías:

- Neuronas biológicas $\leftrightarrow$ Neuronas artificiales

- Conexiones sinápticas ↔ Conexiones ponderadas
- Efectividad de las sinapsis ↔ Peso de las conexiones
- Efecto excitador o inhibidor de una conexión ↔ Signo del peso de una conexión
- Activación → Tasa de disparo ↔ Función de activación → Salida

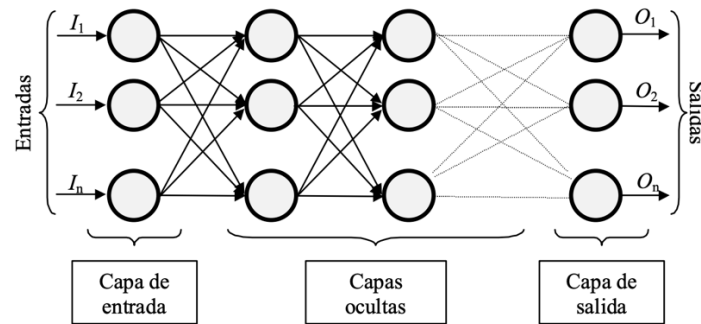
4.5. Ventajas que ofrecen las redes neuronales

Debido a su constitución y a sus fundamentos, las redes neuronales artificiales presentan un gran número de características semejantes a las del cerebro. Por ejemplo, son capaces de aprender de la experiencia, de generalizar de casos anteriores a nuevos casos, de abstraer características esenciales a partir de entradas que representan información irrelevante, etc. Esto hace que ofrezcan numerosas ventajas y que este tipo de tecnología se esté aplicando en múltiples áreas. Entre las ventajas se incluyen:

- ✓ *Aprendizaje Adaptativo.* Capacidad de aprender a realizar tareas basadas en un entrenamiento o en una experiencia inicial.
- ✓ *Auto-organización.* Una red neuronal puede crear su propia organización o representación de la información que recibe mediante una etapa de aprendizaje.
- ✓ *Tolerancia a fallos.* La destrucción parcial de una red conduce a una degradación de su estructura; sin embargo, algunas capacidades de la red se pueden retener, incluso sufriendo un gran daño.
- ✓ *Operación en tiempo real.* Los cálculos neuronales pueden ser realizados en paralelo; para esto se diseñan y fabrican máquinas con hardware especial para obtener esta capacidad.
- ✓ *Fácil inserción dentro de la tecnología existente.* Se pueden obtener chips especializados para redes neuronales que mejoran su capacidad en ciertas tareas. Ello facilitará la integración modular en los sistemas existentes.

4.6. Elementos básicos que componen una red neuronal

En la siguiente imagen se puede ver un esquema general de una red neuronal:



La misma está constituida por neuronas interconectadas y organizadas en tres capas (esto último puede variar). Los datos ingresan por medio de la “capa de entrada”, pasan a través de la “capa oculta” y salen por la “capa de salida”. Cabe mencionar que la capa oculta puede estar constituida por varias capas.

La distribución de neuronas dentro de la red se realiza formando niveles o capas, con un número determinado de dichas neuronas en cada una de ellas. A partir de su situación dentro de la red, se pueden distinguir tres tipos de capas:

- ✓ *De entrada:* es la capa que recibe directamente la información proveniente de las fuentes externas de la red.
- ✓ *Ocultas:* son internas a la red y no tienen contacto directo con el entorno exterior. El número de niveles ocultos puede estar entre cero y un número elevado. Las neuronas de las capas ocultas pueden estar interconectadas de distintas maneras, lo que determina, junto con su número, las distintas topologías de redes neuronales.
- ✓ *De salidas:* transfieren información de la red hacia el exterior.

4.6.1. Función de red o propagación

Esta función se encarga de transformar las diferentes entradas que provienen de la capa anterior en el potencial de la neurona actual. Esta función permite resolver el problema de combinar las entradas simples $(x_1, x_2, x_3, \dots)$ en una sola entrada global, la cual se puede describir de la siguiente forma:

$$input = (x_1 : w_1) * (x_2 : w_2) * \dots * (x_n : w_n)$$

donde “ : ” es el operador que corresponda (por ejemplo: sumatoria, máximo, producto, etc), x_i representa las distintas entradas y w_i los respectivos pesos de conexiones. La multiplicación de las entradas por pesos $x_i w_i$ permite que un valor muy grande de la entrada pueda tener una influencia pequeña, si el peso es pequeño. Por lo general, este operador suele ser la suma ponderada de todas las entradas multiplicadas por sus respectivos pesos.

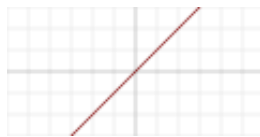
$$y' = \sum_{i=1}^n x_i w_i$$

4.6.2. Función de activación

Esta función combina el potencial que tiene la función de propagación con el estado actual de la neurona para conseguir el estado futuro de ésta (activada o desactivada). Esta función comúnmente es monótona y entre las más usadas están las siguientes:

- Función identidad: es una función de activación muy simple que siempre devuelve como salida la misma entrada.

$$f(x) = x$$



- Función escalón binario: es la función más usada por redes neuronales binarias. No es lineal y es simple de manejar.

$$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$$



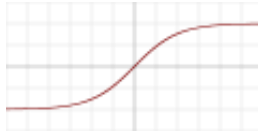
- Función Logística (también conocida como Sigmoido o Softstep): es una de las funciones más usadas para modelar redes neuronales. Su rango es continuo en el intervalo (0, 1), es monótona e infinitamente diferenciable.

$$f(x) = \frac{1}{1 + e^{-x}}$$



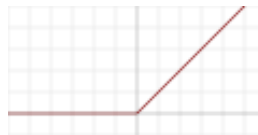
- Función Tangente Hiperbólica: esta función es usada por redes con salidas continuas. Por ejemplo, el Perceptrón multicapa con retropropagación, ya que su algoritmo de aprendizaje necesita una función derivable. Es monótona y se aproxima a la función identidad alrededor del origen.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- Función rectificadora (ReLU – Rectified Linear Unit): esta función la introdujo por primera vez Hahnloser en el año 2000 con motivaciones biológicas y matemáticas. Se viene usando en redes convolucionales más que la función logística sigmoide y es más práctica que la función tangente hiperbólica. Es una de las funciones más populares para el Deep Learning.

$$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$$



- Función Softplus: es una aproximación suavizada de la función rectificadora.

$$f(x) = \ln(1 + e^x)$$



4.6.3. Función de pérdida

La función de pérdida es uno de los parámetros necesarios para compilar un modelo. Algunas de las funciones más conocidas que nos proporciona la biblioteca Keras son:

- ✓ `mean_squared_error (y_true, y_pred)`: Calcula el error cuadrático medio.
- ✓ `mean_absolute_error / mae`: Calcula el error medio absoluto.

- ✓ `binary_crossentropy`: Calcula la entropía cruzada en problemas de clasificación binaria. También es conocida como pérdida logarítmica.

4.6.4. Regularizadores

Los regularizadores permiten aplicar penalizaciones en las capas durante el proceso de optimización. Estas penalizaciones se incorporan a la función de pérdida que la red optimiza con el fin de evitar que la red neuronal aprenda “demasiado bien” los datos de entrenamiento; es decir, para prevenir el sobreajuste (overfitting). En otras palabras, agrega un “castigo o penalización” al valor de la función de pérdida si los pesos del modelo crecen demasiado o si se vuelven muy complejos. Por ejemplo:

```
model.add(Dense(64, input_dim=64, W_regularizer=12 (0.01),  
activity_regularizer=activity_12(0.01)))
```

4.6.5. Optimizadores

El optimizador es otro de los valores necesarios para compilar un modelo (junto con la función de pérdida). Ajustan los pesos de la red neuronal durante el entrenamiento para que la función de pérdida se minimice lo más rápido y eficientemente posible. Usan técnicas matemáticas basándose en los gradientes calculados con backpropagation. Los principales optimizadores son:

- ✓ SGD: aproximación estocástica del método de gradiente descendiente.
- ✓ Adagrad: algoritmo basado en gradiente que adapta el ratio de aprendizaje a los parámetros, realizando grandes actualizaciones cuando éstos son infrecuentes, y pequeñas actualizaciones cuando éstas son frecuentes.
- ✓ Adadelta: es una extensión del algoritmo Adagrad.
- ✓ Adam: algoritmo basado en el gradiente de primer orden de funciones estocásticas objetivas. Es un método sencillo de aplicar, es eficiente computacionalmente, necesita poca capacidad de memoria y es muy usado en problemas con grandes cantidades de datos y/o parámetros.

Por ejemplo:

```
from keras.optimizers import Adam  
model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])
```

4.6.6. Callbacks

Son funciones que pueden ser aplicadas en ciertos momentos del proceso de entrenamiento de la red neuronal. Estos callbacks pueden ser utilizados para observar los estados internos de la red, así como las estadísticas del modelo que

se está entrenando, es decir, sirven para controlar, monitorear o modificar el entrenamiento en tiempo real.

Estas funciones permiten guardar el modelo cuando mejora, detener el entrenamiento si no mejora (EarlyStopping), ajustar la tasa de aprendizaje automáticamente y registrar métricas para visualización (como TensorBoard).

Para utilizarlos, basta con pasar como argumento a la función `fit(...)`, una lista de métodos que serán llamados en cada fase del entrenamiento. Entre los principales Callbacks se encuentran los siguientes:

- ✓ `BaseLogger ( )`: se aplica en todo modelo de Keras y acumula las medias de las métricas que se han definido para ser monitoreadas por época.
- ✓ `Callback ( )`: clase base abstracta utilizada para crear nuevos callbacks. Como parámetros, recibe los propios para configurar el entrenamiento (número de ciclos, verbosidad, ...), así como la instancia del modelo que se quiere entrenar.
- ✓ `ProgbarLogger ( )`: Callback que muestra métricas en la salida estándar (stdout).
- ✓ `History ( )`: Callback aplicado en cada modelo Keras que almacena eventos en un objeto tipo `History`.
- ✓ `ModelCheckpoint (...)`: almacena el modelo después de cada época. Como argumento recibe el fichero donde deja la información, la cantidad de elementos a monitorear, el modo de verbosidad, un booleano para decidir si sobrescribir o no el mejor modelo, el modo, y otro booleano para guardar solo pesos o información de todo el modelo.

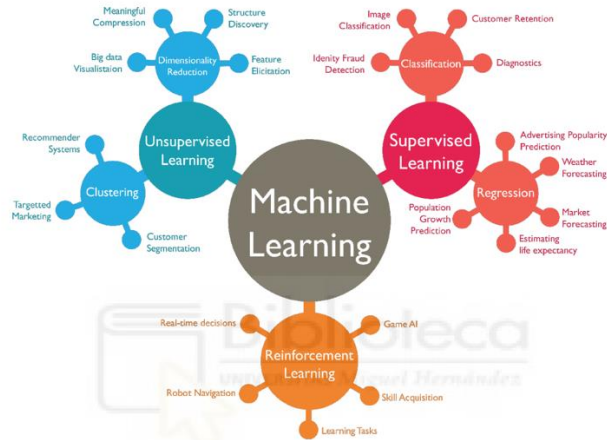
Por ejemplo:

```
from keras.callbacks import EarlyStopping, ModelCheckpoint
early = EarlyStopping(patience=3, restore_best_weights=True)
checkpoint = ModelCheckpoint("mejor_modelo.h5", save_best_only=True)

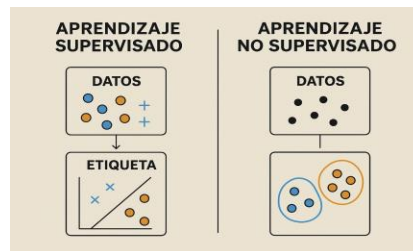
model.fit(X_train, y_train, epochs=50, validation_data=(X_val,
y_val), callbacks=[early, checkpoint])
```

4.7. Tipos de aprendizaje en redes neuronales

El aprendizaje supervisado y no supervisado son dos de los principales enfoques en el ámbito de la inteligencia artificial y el aprendizaje automático. La forma más simple de diferenciar ambos aprendizajes es conociendo la manera en que son entrenados los modelos y el tipo de datos de entrenamiento que usan los algoritmos.



El aprendizaje supervisado utiliza datos de entrenamiento etiquetados por el humano, mientras que el aprendizaje no supervisado no usa ese tipo de datos, es decir, los modelos de aprendizaje supervisados tienen una comprensión básica de cuáles deben ser los valores de salidas correctos.



En los modelos de aprendizaje supervisado, el algoritmo usa un conjunto de datos de muestra para entrenarse y hacer predicciones, y se ajusta de forma iterativa para minimizar los errores. Estos conjuntos de datos están etiquetados para aportar contexto, lo que proporciona los valores de salida deseados para que el modelo pueda dar la respuesta correcta. Por el contrario, en los modelos de aprendizaje no supervisados, el algoritmo funciona de forma independiente para aprender la estructura inherente de los datos sin necesidad de indicarle directrices o instrucciones específicas, es decir, solo hay que proporcionar los datos de entrada y dejar que el algoritmo identifique algún patrón natural en el conjunto de datos. Este tipo de aprendizaje es el más comparable al proceso que tiene lugar en el cerebro humano cuando aprende cosas nuevas.

Dicho de otro modo, los modelos de aprendizaje supervisado se centran en aprender las relaciones entre los datos de entrada y las salidas. Por ejemplo, un modelo supervisado se podría usar para predecir las horas de vuelo en función de parámetros específicos como las condiciones meteorológicas, el tráfico aéreo, etc. Por otra parte, el aprendizaje no supervisado es más útil para descubrir nuevos patrones o relaciones en datos sin procesar ni etiquetar. Por ejemplo, un modelo no supervisado se podría usar para identificar grupos de compradores que compren productos relacionados juntos para sugerir otros artículos que se puedan recomendar a otros clientes similares.

4.8. Principales topologías de la redes neuronales

La topología o arquitectura de una red neuronal consiste en la organización y disposición de las neuronas en la misma, formando capas o agrupaciones de neuronas más o menos alejadas de la entrada y salida de dicha red. En este sentido, los parámetros fundamentales de la red son: el número de capas, el número de neuronas por capa, el grado de conectividad y el tipo de conexiones entre neuronas.

4.8.1. Redes monocapa

En las redes monocapa, se establecen conexiones entre las neuronas que pertenecen a la única capa que constituye la red. Las redes monocapas se utilizan generalmente en tareas relacionadas con lo que se conoce como autoasociación (generar información de entrada que se presenta a la red de forma incompleta o distorsionada).

4.8.2. Redes multicapas

Las redes multicapas son aquellas que disponen de un conjunto de neuronas agrupadas en varios (2, 3, etc.) niveles o capas. En estos casos, una forma para distinguir la capa a la que pertenece una neurona, consistiría en fijarse en el origen de las señales que recibe a la entrada y el destino de la señal de salida. Normalmente, todas las neuronas de una capa reciben señales de entrada desde otra capa anterior (la cual está más cerca a la entrada de la red), y envían señales de salida a una capa posterior (que está más cerca a la salida de la red). A estas conexiones se las denomina conexiones hacia delante o feedforward. Sin embargo, en un gran número de estas redes también existe la posibilidad de conectar la salida de las neuronas de capas posteriores a la entrada de capas anteriores; a estas conexiones se las denomina conexiones hacia atrás o feedback.

Estas dos posibilidades permiten distinguir entre dos tipos de redes con múltiples capas: las redes con conexiones hacia adelante o redes feedforward, y las redes que disponen de conexiones tanto hacia adelante como hacia atrás o redes feedforward/feedback.

4.8.3. Conexión entre neuronas

La conectividad entre los nodos de una red neuronal está relacionada con la forma en que las salidas de las neuronas están canalizadas para convertirse en entradas de otras neuronas. La señal de salida de un nodo puede ser una entrada de otro elemento de proceso, o incluso ser una entrada de sí mismo (conexión auto recurrente).

4.9. Algunas aplicaciones de la redes neuronales

Las redes neuronales pueden utilizarse en un gran número y variedad de aplicaciones, tanto comerciales como militares, tales como:

En la Biología:

- ✓ Aprender más acerca del cerebro y otros sistemas.
- ✓ Obtención de modelos de la retina.

En la empresa:

- ✓ Evaluación de probabilidad de formaciones geológicas y petrolíferas.
- ✓ Identificación de candidatos para posiciones específicas.
- ✓ Explotación de bases de datos.
- ✓ Optimización de plazas y horarios en líneas de vuelo.
- ✓ Optimización del flujo del tránsito controlando convenientemente la temporización de los semáforos.
- ✓ Reconocimiento de caracteres escritos.
- ✓ Modelado de sistemas para automatización y control.

En el medio ambiente:

- ✓ Analizar tendencias y patrones.
- ✓ Previsión del tiempo.

En Finanzas:

- ✓ Previsión de la evolución de los precios.
- ✓ Valoración del riesgo de los créditos.
- ✓ Identificación de falsificaciones.
- ✓ Interpretación de firmas.

En Manufacturación:

- ✓ Robots automatizados y sistemas de control (visión artificial y sensores de presión, temperatura, gas, etc.).
- ✓ Control de producción en líneas de procesos.
- ✓ Inspección de la calidad.

En la Medicina:

- ✓ Analizadores del habla para ayudar en la audición de sordos profundos.
- ✓ Diagnóstico y tratamiento a partir de síntomas y/o de datos analíticos (electrocardiograma, encefalogramas, análisis sanguíneo, etc.).
- ✓ Monitorización en cirugías.

- ✓ Predicción de reacciones adversas en los medicamentos.
- ✓ Entendimiento de la causa de los ataques cardíacos.

En la Milicia:

- ✓ Clasificación de las señales de radar.
- ✓ Creación de armas inteligentes.
- ✓ Optimización del uso de recursos escasos.
- ✓ Reconocimiento y seguimiento en el tiro al blanco.

4.10. Un poco sobre la biblioteca Keras

Keras es una biblioteca de código abierto, de alto nivel para el desarrollo y entrenamiento de modelos de aprendizaje automático y de aprendizaje profundo. Fue desarrollada por François Chollet y es parte del ecosistema de TensorFlow.

Se caracteriza por ser una biblioteca sencilla de usar y flexible, ya que proporciona una interfaz simple y amigable para construir redes neuronales, lo que permite a los desarrolladores crear y entrenar modelos de aprendizaje automático de manera rápida y eficiente.

Keras tiene varias características que la hacen ideal para la construcción de modelos de aprendizaje automático. Algunas de estas características incluyen:

- Fácil de usar: Keras tiene una API simple y fácil de usar.
- Rendimiento: Keras es rápido y eficiente, lo que la hace ideal para el desarrollo de modelos de aprendizaje automático complejos.
- Compatibilidad con frameworks: Keras es una librería de alto nivel que se puede utilizar con TensorFlow, JAX y PyTorch.
- Soporta tanto redes neuronales convolucionales como recurrentes, incluso combinación de ambas.

Keras se basa en un concepto llamado “capas”, las cuales constituyen los bloques fundamentales para construir modelos de aprendizaje automático. Keras incluye dos tipos de modelos, según las necesidades del desarrollador:

- *Modelo secuencial*, el cual es adecuado para apilar un conjunto de capas, donde cada una de ellas tiene exactamente un tensor de entrada y un tensor de salida. Este modelo no se considera adecuado cuando el modelo o al menos una de sus capas tiene múltiples entradas o múltiples salidas y cuando quieres crear una topología no lineal.

Los métodos básicos del modelo secuencial son:

- ✓ `compile(self, optimizar, los, metrics=[], sample_weight_mode=None)`

Calcula automáticamente la función gradiente y prepara el modelo para una forma concreta del entrenamiento.

- ✓ `fit(self, x, y, batch_size=32, nb_epoch=10, verbose=1, callbacks=[], validation_split=0.0, validation_data=None, shuffle=True, class_weight=None, sample_weight=None):` Entrena el modelo para un número fijado de ciclos (`nb_epoch`), con actualizaciones de gradientes cada `batch_size` ejemplos.
- ✓ `evaluate(self, x, y, batch_size=32, verbose=1, sample_weight=None):` Calcula la función de pérdida dados unos datos de entrada, lote por lote.
- ✓ `predict(self, x, batch_size=32, verbose=0):` Genera predicciones para unos valores de entrada.

- *Modelo funcional*, el cual es más flexible que el modelo secuencial, ya que puede manejar modelos con topologías no lineales, puede manejar capas compartidas e incluso múltiples entradas y/o salidas.

Algunas de las capas más utilizadas con Keras son las siguientes:

- *Capas de convolución*, las cuales son usadas para procesar imágenes y otros tipos de datos con representaciones en 2D.
- *Capas de pooling*, las cuales son usadas para reducir la dimensionalidad de los datos.
- *Capas densas*, las cuales son las más común es la construcción de modelos de aprendizaje automático, aprende las relaciones entre los datos para poder llevar a cabo la tarea objetivo.
- *Capas LSTM (Long Short-Term Memory)*, las cuales son usadas para procesar datos secuenciales, como series temporales y textos.
- *Capas dropout*, las cuales son usadas para regularizar el modelo y evitar sobreajuste, es decir, se eliminan un conjunto de neuronas de manera aleatoria durante el entrenamiento.
- *Capas de embedding*, las cuales son usadas para convertir datos en representaciones numéricas densas, lo que permite que el modelo aprenda relaciones entre los datos y generalmente se utilizan en tareas de procesamiento de lenguaje natural (NLP).

4.11. Redes neuronales especiales

Todas las neuronas tienen pesos, sesgos, reciben una entrada con la que realiza un producto escalar y sobre la que aplica una función de activación y también cuenta con función de pérdida.

4.11.1. Redes neuronales convolucionales

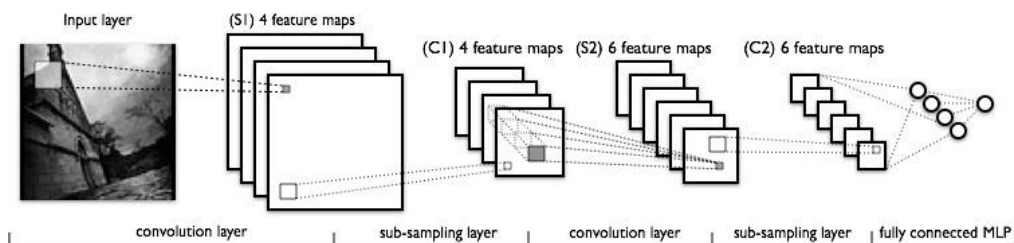
Se usan principalmente para resolver el mayor problema que tienen las redes neuronales ordinarias “el tratamiento de imágenes”. A pesar de que con una red neuronal estándar es posible tratar imágenes, en cuanto éstas aumentan su tamaño y calidad, el tratamiento se vuelve muy intratable. Al tratarse de neuronas totalmente conectadas, se provocaría un desperdicio de recursos, así como un rápido sobreajuste.

Las redes neuronales convolucionales trabajan dividiendo y modelando la información en partes más pequeñas, y combinando esta información en las capas más profundas de la red. Por ejemplo, en el caso de tratamiento de una imagen, las primeras capas tratarán de detectar los bordes de las figuras. Las siguientes capas buscan combinar los patrones de detección de bordes para conseguir formas más simples, y aplicar patrones de posición de objetos, iluminación. Por último, en las últimas capas se trata de hacer coincidir la imagen con todos los patrones descubiertos, para conseguir una predicción final de la suma de todos ellos.

De esta forma, las redes neuronales convolucionales consiguen modelar una gran cantidad de datos, dividiendo previamente el problema en partes más pequeñas con el fin de conseguir predicciones más sencillas y precisas.

En general, todas las redes neuronales convolucionales están formadas por una estructura compuesta por tres tipos de capas:

- Capa convolucional: esta capa le da el nombre a la red. En vez de usar la multiplicación de matrices como en las redes neuronales ordinarias, se aplica una operación llamada convolución. Esta operación recibe como entrada una imagen, y sobre ella aplica un filtro que nos devuelve un mapa con sus características, reduciendo de esa manera el tamaño de sus parámetros. La convolución usa tres ideas que ayudan a mejorar cualquier sistema sobre el que se aplique machine learning:
 - a) Interacciones dispersas: al aplicar un filtro de menor tamaño sobre la entrada, se reduce el número de parámetros y cálculos.
 - b) Parámetros compartidos: compartir parámetros entre los distintos tipos de filtros ayuda a mejorar la eficiencia del sistema.
 - c) Representaciones equivariantes: si las entradas cambian entonces las salidas cambian de forma similar.



La biblioteca Keras presenta diversas funciones para desarrollar capas convolucionales según las dimensiones de su entrada. Como ejemplo, la más usada es la siguiente:

```
Convolution2D(nb_filter, nb_row, nb_col, init='glorot_uniform',  
activation=None, weights=None, border_mode='valid', subsample=(1, 1),  
dim_ordering='default', W_regularizer=None, b_regularizer=None,  
activity_regularizer=None, W_constraint=None, b_constraint=None, bias=True)
```

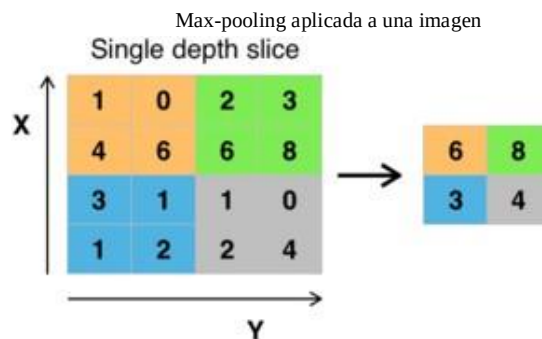
Un ejemplo más específico donde se define una capa convolucional de dos dimensiones, con 32 mapeadores de tamaño 5x5 y una función de activación rectificadora, sería:

```
model.add(Convolution2D(32, 5, 5, border_mode='valid', activation='relu'))
```

- **Capa de reducción o pooling:** esta capa se coloca generalmente después de la capa convolucional. Es la encargada de reducir la cantidad de parámetros a analizar reduciendo las dimensiones espaciales, quedándose de esta forma con las características más comunes. La operación que lleva a cabo esta capa también se conoce como reducción de muestreo, ya que la reducción de tamaño implica también una pérdida de información. Sin embargo, para una red neuronal, esta pérdida de información puede ser beneficioso ya que:

- a) La reducción del tamaño provoca una menor sobrecarga de cálculos en las próximas capas de la red.
- b) Reduce habitualmente el sobreajuste.

La operación que se usa en esta capa es “max-pooling”, que divide la imagen de entrada en un conjunto de rectángulos, y respecto a cada uno de ellos, se queda con el valor máximo.



La biblioteca Keras ofrece un gran número de funciones para desarrollar capas pooling en redes convolucionales, dependiendo de las dimensiones de la entrada. Como ejemplo, una función representativa sería:

```
MaxPooling2D(pool_size=(2, 2), strides=None, border_mode='valid',  
dim_ordering='default')
```

- Capa clasificadora totalmente conectada: una vez que la capa ha pasado tanto por las capas convolucionales como por las capas de pooling y se han extraído sus características más destacadas, los datos llegan a la fase de clasificación. Para ello, las redes convolucionales normalmente usan capas completamente conectadas en las que cada píxel se trata como una neurona independiente. Las neuronas de esta fase funcionan como las de un perceptrón multicapa, donde la salida de cada neurona se calcula multiplicando la salida de la capa anterior por el peso de la conexión, y aplicando a este dato una función de activación.

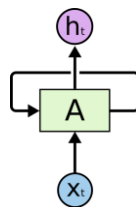
4.11.2. Redes neuronales recurrentes

Los seres humanos no empezamos a pensar desde cero a cada segundo. Por ejemplo, mientras leemos entendemos cada palabra basándonos en el contexto que forman las palabras previas. No desperdiciamos las ideas anteriores, sino que éstas persisten en nuestra memoria.

Las redes neuronales tradicionales no cuentan con esta capacidad y es su mayor defecto. Por ejemplo, imaginemos el caso en el que queremos clasificar qué clase de evento está pasando en un punto de una película. No está claro cómo una red neuronal tradicional podría usar razonadamente las escenas anteriores para predecir las siguientes.

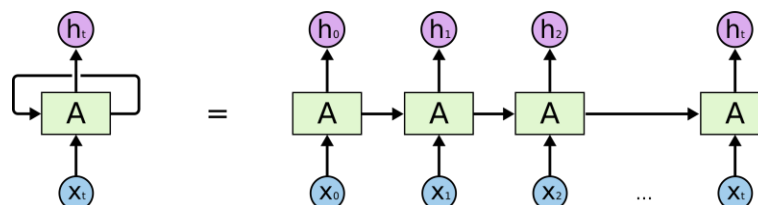
Aquí es donde entran en juego las redes neuronales recurrentes, ya que estas resuelven este problema. Su principal característica es que son redes con bucles que permiten que la información persista.

Ejemplo básico de una red neuronal recurrente



En la imagen anterior, se puede observar como una parte de la red neuronal, **A**, recibe una entrada x y devuelve una salida h . El bucle permite que la información pase de un ciclo de la red a la siguiente.

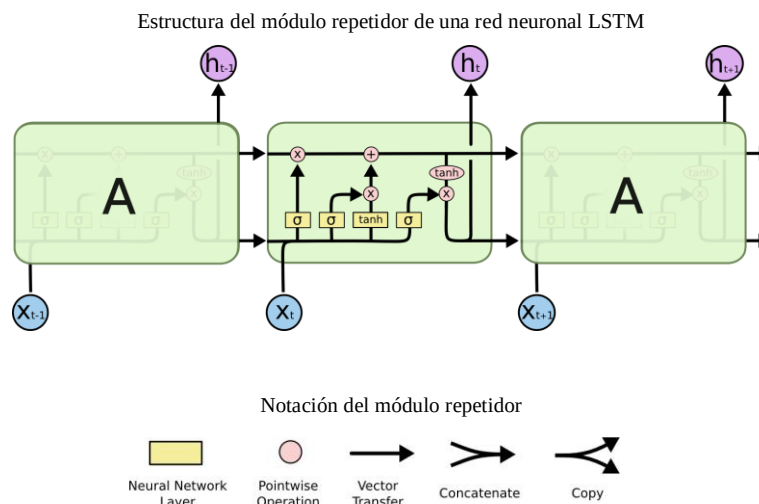
Una red neuronal recurrente puede ser creada usando múltiples copias de la misma red, pasando el mensaje o la salida al nodo sucesor. Desenrollando el bucle, tendríamos algo parecido a lo que se puede observar en la siguiente figura:



Esta forma de cadena hace prever que las redes neuronales recurrentes están relacionadas con secuencias y listas. En los últimos años, se ha aplicado este tipo de redes de forma exitosa para resolver problemas como reconocimiento de discursos, modelación de lenguajes, traducciones, etc. Un factor clave en este éxito es el uso de redes LSTM, las cuales son un tipo muy especial de redes neuronales recurrentes.

Las redes Long Short Term Memory , también conocidas como redes LSTM, son un tipo de redes neuronales recurrentes descritas por primera vez en 1997 por Hochreiter y Schmidhuber, donde su característica principal es que son capaces de aprender una larga lista de dependencias en largos períodos de tiempo.

Todas las redes neuronales recurrentes tienen forma de cadena al repetir sus módulos. En las RNN estándar, el módulo repetidor tiene una estructura muy simple, con una sola capa, en cambio que en las redes LSTM también tienen forma de cadena, pero el módulo repetidor tiene cuatro capas en vez de una sola.



Las claves de las redes LSTM se encuentran en el estado de la celda, es decir, en la línea horizontal que recorre la parte superior del diagrama, ya que permite a la información fluir por toda la red con pocas interacciones, sin ser apenas alteradas. La red también tiene la habilidad de borrar o añadir información al estado de la celda mediante estructuras llamadas puertas. Estas puertas son una manera de permitir opcionalmente a la información circular o no, y se componen de una capa sigmoide y una operación de multiplicación. Las salidas de la capa sigmoide alternan entre 0 y 1, describiendo si la información fluye o no. Este tipo de redes tienen 3 de este tipo de puertas con el fin de proteger y controlar el estado de la celda.

Las redes neuronales LSTM siguen una serie de pasos para poder decidir qué información se va ir almacenando o borrando:

- Paso 1. Decidir qué información despreciar

Esta decisión la toma la capa sigmoide, que devuelve una salida igual a 1 si se busca mantener la información, o 0 si queremos borrar dicha información. Por ejemplo: si se quiere predecir una palabra basándose en las anteriores, el estado de la celda puede almacenar el género del sujeto para usar los pronombres adecuados. Cuando nos encontramos un nuevo sujeto, olvidaremos el género del anterior ya que no será de utilidad.

- Paso 2. Decidir qué información se va a almacenar

Este paso consta de dos partes. Primero, una capa sigmoide decide qué valores son actualizados. Después una capa tangente hiperbólica crea un vector con los nuevos valores candidatos que pueden ser añadidos al estado. En el mismo contexto del ejemplo del paso 1, se decidiría que queremos añadir el género del nuevo sujeto al estado de la celda, para reemplazar el viejo que queremos olvidar.

- Paso 3. Actualizar el estado de la celda.

En el paso anterior se decide qué hacer, y ahora toca hacerlo. Se multiplica el viejo estado por la salida de la capa sigmoide del paso 1, olvidando así las cosas que se decidieron olvidar. A esto, se le suma el producto de la salida de la capa sigmoide y de la capa tangente hiperbólica del paso 2, con el fin de conocer los nuevos valores candidatos, escalados según la decisión de actualizar el estado de la celda. Siguiendo el contexto del mismo ejemplo de los pasos anteriores, ahora es cuando se desecha la información sobre el género del viejo sujeto y se añade la nueva información, como se decidió en los pasos anteriores.

- Paso 4. Decidir la salida basándose en el estado de la celda, pero filtrada.

Primero, mediante una capa sigmoide, se decide qué partes del nuevo estado de la celda se van a sacar como salida. Después, se pasa el estado de la celda, por una capa tangente hiperbólica para tener valores entre -1 y 1, y se multiplica por la salida de la capa sigmoide para sacar solo las partes que se decidan. En el mismo contexto del ejemplo de los pasos anteriores, una vez que se encuentre el nuevo sujeto, se sacará

información acerca de cómo puede ser si éste es plural o singular, para después poder conjugar correctamente el verbo que sigue.

A continuación, presento dos ejemplos prácticos en los que se combina TensorFlow con Keras para llevar a cabo algunas tareas clásicas de IA. En cada ejemplo, se divide el proceso de crear y entrenar la red neuronal en 4 fases:

- Fase 1 (preparación del entorno): se realizan los imports necesarios para que el código se ejecute correctamente.
- Fase 2 (preparación de datos): descargamos los datos y los preprocesamos para poder usarlos.
- Fase 3 (creación del modelo): se define el modelo a utilizar.
- Fase 4 (entrenamiento y evaluación del modelo): se entrena y se evalúa el modelo usado.

Ejemplo 1: Visión por computador

En primer lugar, vamos a usar el conjunto de datos MNIST para clasificar imágenes de dígitos escritos a mano. El objetivo es entrenar un modelo que pueda detectar el número que aparece en cada imagen.

El primer paso es importar las librerías que usaremos.

```
import numpy as np
import keras
from keras import layers
```

A continuación, normalizamos los datos. Específicamente, reescalamos los grises a una escala con 255 valores.

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
print("x_train shape:", x_train.shape)
print(x_train.shape[0], "train samples")
print(x_test.shape[0], "test samples")

num_classes = 10
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

Ahora, creamos el modelo que usaremos para clasificar el dígito de la imagen. Específicamente lo haremos por medio de un modelo secuencial.


```

input_shape = (28, 28, 1)

model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax"),
    ]
)

model.summary()

```

Finalmente, entrenamos el modelo y lo evaluamos:

```

batch_size = 128
epochs = 15

model.compile(loss="categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])

model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_split=0.1)

score = model.evaluate(x_test, y_test, verbose=0)
print ("Test loss:", score[0])
print ("Test accuracy:", score[1])

```

Explicación detallada:

Paso 1: Importar librerías

```

import numpy as np
import keras
from keras import layers

```

- `numpy as np`: importa la biblioteca NumPy, usada para manipular arrays (matrices, vectores, etc.).
- `import keras`: importa Keras, un API de alto nivel para construir y entrenar modelos de redes neuronales.
- `from keras import layers`: permite acceder fácilmente a las capas (Dense, Conv2D, etc.) de redes neuronales.

Paso 2: Cargar y normalizar el dataset MNIST

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

- Descarga el dataset MNIST de Keras, que contiene imágenes de dígitos escritos a mano (0 al 9).

- `x_train, y_train`: son las imágenes y etiquetas para entrenamiento.
- `x_test, y_test`: son las imágenes y etiquetas para pruebas (evaluación).

```
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255
```

- Convierte los valores de los píxeles a tipo `float32` y los normaliza dividiendo entre 255 (el valor máximo de un píxel en escala de grises).
- Esto asegura que los valores estén entre 0 y 1, lo que mejora el entrenamiento del modelo.

```
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
```

- Agrega una dimensión extra al final del array: `-1` representa el canal de color.
- Ya que las imágenes de MNIST son en blanco y negro, se usa 1 canal (forma final: `28x28x1`).
- Esto convierte los datos a formato adecuado para una red convolucional.

```
print("x_train shape:", x_train.shape)
print(x_train.shape[0], "train samples")
print(x_test.shape[0], "test samples")
```

Esta parte muestra cuántas imágenes de entrenamiento y prueba hay, y la forma de las imágenes. Por ejemplo: `(60000, 28, 28, 1)` representa 60,000 imágenes de 28x28 píxeles, con 1 canal.

Paso 3: Convertir etiquetas a codificación one-hot

```
num_classes = 10
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

- Define que hay 10 clases (los dígitos 0 al 9).
- Convierte las etiquetas (por ejemplo, 5) a vectores de 10 posiciones:
Ejemplo: `5` → `[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]`
Esto es necesario para usar `categorical_crossentropy` como función de pérdida.

Paso 4: Construcción del modelo secuencial

```
input_shape = (28, 28, 1)
```

Esta línea define la forma de entrada: imágenes de 28x28 píxeles con 1 canal.

```
model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
```

```

        layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax"),
    ]
)

```

En esta parte se define la estructura de la red neuronal:

1. `keras.Input(shape=...)`: define el tamaño de entrada.
2. `Conv2D(32, kernel_size=(3, 3), activation="relu")`:
 - Crea 32 filtros (detectores de patrones) de tamaño 3x3.
 - Aplica función de activación ReLU.
3. `MaxPooling2D(pool_size=(2, 2))`: reduce el tamaño de la imagen a la mitad (submuestreo).
4. `Conv2D(64, ...)` y `MaxPooling2D(...)`: otra capa convolucional y de agrupación más profunda.
5. `Flatten()`: aplanar la salida 2D a un vector 1D para poder usar capas densas.
6. `Dropout(0.5)`: apaga aleatoriamente el 50% de las neuronas durante el entrenamiento (evita sobreajuste).
7. `Dense(num_classes, activation="softmax")`: capa final con 10 neuronas, una por clase, usando softmax para predecir la probabilidad de cada dígito.

```
model.summary()
```

Esta línea permite mostrar una descripción del modelo: número de capas, parámetros entrenables y forma de entrada/salida.

Paso 5: Entrenar y evaluar el modelo

```
batch_size = 128
epochs = 15
```

- `batch_size`: número de muestras usadas por iteración durante el entrenamiento.
- `epochs`: número de veces que se procesará todo el conjunto de entrenamiento.

```
model.compile(loss="categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])
```

En esta parte se compila el modelo, indicando:

- `loss="categorical_crossentropy"`: es la función de pérdida para clasificación multiclase.
- `optimizer="adam"`: optimizador eficiente para minimizar la pérdida.

- o `metrics=["accuracy"]`: métrica a seguir durante entrenamiento y validación.

```
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_split=0.1)
```

- Entrena el modelo con los datos de entrenamiento.
- Usa el 90% para entrenar y el 10% para validar (`validation_split=0.1`).
- Muestra el progreso por epoch.

```
score = model.evaluate(x_test, y_test, verbose=0)
print ("Test loss:", score[0])
print ("Test accuracy:", score[1])
```

- Evalúa el modelo ya entrenado con los datos no vistos (`x_test, y_test`).
- `score[0]`: pérdida total en prueba.
- `score[1]`: exactitud o porcentaje de aciertos.

Ejemplo 2. Procesamiento de lenguaje natural (NLP)

Para este ejemplo usaremos el conjunto de datos imdb para clasificar reseñas como positivas o negativas.

El primer paso es importar las librerías que usaremos.

```
import numpy as np
import keras
from keras import layers
```

Ahora cargamos los datos y la única transformación que se llevará a cabo es la conversión de los textos en secuencias. Esto es necesario porque usaremos el modelo de capas LSTM.

```
max_features = 20000
maxlen = 200

(x_train, y_train), (x_val_test, y_val_test) =
keras.datasets.imdb.load_data(
    num_words=max_features
)
x_val = x_val_test[:int (len(x_val_test)/2)]
y_val = y_val_test[:int (len(y_val_test)/2)]
x_test = x_val_test[int (len(x_val_test)/2):]
y_test = y_val_test[int (len(y_val_test)/2):]
print(len(x_train), "Training sequences")
print(len(x_val), "Validation sequences")
print(len(x_val), "Test sequences")
```

```
x_train = keras.utils.pad_sequences(x_train, maxlen=maxlen)
x_val = keras.utils.pad_sequences(x_val, maxlen=maxlen)
x_test = keras.utils.pad_sequences(x_test, maxlen=maxlen)
```

A continuación, una vez que tenemos los datos en el formato correcto, creamos el modelo de la siguiente manera:

```
inputs = keras.Input(shape=(None,), dtype="int32")
x = layers.Embedding(max_features, 128)(inputs)
x = layers.Bidirectional(layers.LSTM(64, return_sequences=True))(x)
x = layers.Bidirectional(layers.LSTM(64))(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.summary()
```

Y finalmente entrenamos y evaluamos el modelo:

```
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
model.fit(x_train, y_train, batch_size=32, epochs=2,
validation_data=(x_val, y_val))

score = model.evaluate(x_test, y_test, verbose=0)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```